РУКОВОДСТВО РАЗРАБОТЧИКА

Проект bookstore

МДК.04.01 Внедрение, поддержка компьютерных систем и обеспечение качества функционирования компьютерных систем

1. Общие сведения о проекте

Параметр	Значение
Название	Книжный магазин «Буквоед»
Версия	1.0.0
Стек	Python 3.12, Django 5.1, DRF 3.15, PostgreSQL 16
Репозиторий	локальный git-репозиторий проекта

2. Структура проекта

```
bookstore/
├── apps/
│   ├── accounts/   models.py  admin.py
│   ├── catalog/    models.py  serializers.py  views.py  urls.py  admin.py
│   ├── clients/    models.py  serializers.py  views.py  urls.py  admin.py
│   ├── orders/     models.py  serializers.py  views.py  urls.py  admin.py
│   └── reports/    services.py  views.py  urls.py
├── config/
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├── .env.example
├── manage.py
├── requirements.txt
└── README.md
```

3. Установка и настройка

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt

copy .env.example .env      # заполнить SECRET_KEY и параметры БД
python manage.py migrate
python manage.py seed       # демонстрационные данные
python manage.py runserver
```

Для PostgreSQL раскомментируйте соответствующий блок DATABASES в config/settings.py и заполните переменные в .env. Учебный запуск использует SQLite, чтобы проект стартовал без установки СУБД.

4. Описание API

Метод	URL	Назначение
GET	/api/	корневой список эндпоинтов
GET	/api/books/	список позиций
POST	/api/books/	создать позицию
GET	/api/books/<built-in function id>/	одна позиция
PATCH	/api/books/<built-in function id>/	изменить позицию
GET	/api/clients/	список клиентов

Метод	URL	Назначение
GET	/api/orders/	список заказов
GET	/api/reports/kpi/	ключевые показатели
POST	/api/auth/login/	получить JWT-токен

Пример запроса и ответа:

```
curl http://127.0.0.1:8000/api/books/ \
     -H "Authorization: Bearer <ACCESS_TOKEN>"

{
  "count": 6,
  "next": null,
  "previous": null,
  "results": [
    {"id": 1, "name": "Пример", "price": "2500.00", "status": "active"}
  ]
}
```

5. Аутентификация (JWT)

Токен выдаётся по логину и паролю и передаётся в заголовке Authorization. Время жизни access-токена — 60 минут, refresh — 1 сутки.

```
POST /api/auth/login/
{"username": "admin", "password": "admin12345"}

{
  "refresh": "eyJhbGciOiJIUzI1NiIs...",
  "access": "eyJhbGciOiJIUzI1NiIs..."
}

POST /api/auth/refresh/
{"refresh": "eyJhbGciOiJIUzI1NiIs..."}
```

6. Тестирование

Тесты запускаются командой python manage.py test. Покрыты бизнес-логика каскада статусов и доступность API.

```
Ran 5 tests in 0.013s

OK
```

Чтобы добавить тест, создайте класс в apps/<модуль>/tests.py, унаследуйте его от TestCase и опишите сценарий в методе test_*.

7. Миграции

```
python manage.py makemigrations
python manage.py migrate
python manage.py migrate orders 0001   # откат до первой миграции
```

Правило: после изменения моделей обязательно создаётся новая миграция; уже применённые миграции не редактируются.

8. Развёртывание

Локально проект запускается через runserver. Для продуктивного развёртывания используется gunicorn + nginx, статика собирается командой collectstatic, секреты хранятся в переменных окружения.

9. Частые проблемы

Проблема	Решение
No module named 'apps.X'	проверить apps.py и наличие __init__.py
UnicodeDecodeError	пересоздать БД с кодировкой UTF-8
401 Unauthorized	проверить срок жизни и формат токена
relation does not exist	применить миграции
no such table	выполнить migrate, проверить путь к БД

10. Контакты

Автор проекта: Лукашин Елисей Андреевич, группа ИСП-211. Документация: https://docs.djangoproject.com/